

Infrastructure / SRE Intern Assignment

Candidate: Jayesh

Project: Mission-Critical Incident Management System (IMS)

Role: Infrastructure / SRE Intern

Submission Date: 5 May 2026

GitHub Repository: <https://github.com/jayesh9747/zeotap-SRE-assignment.git>

1. Project Overview

The Incident Management System is a Dockerized full-stack application built to ingest high-volume operational signals, group noisy failure events into incidents, store raw evidence, and support an operator workflow from OPEN to CLOSED with mandatory Root Cause Analysis.

The first qualification criteria is a running application, so the project is packaged with one-command Docker Compose setup.

2. One-Line Setup

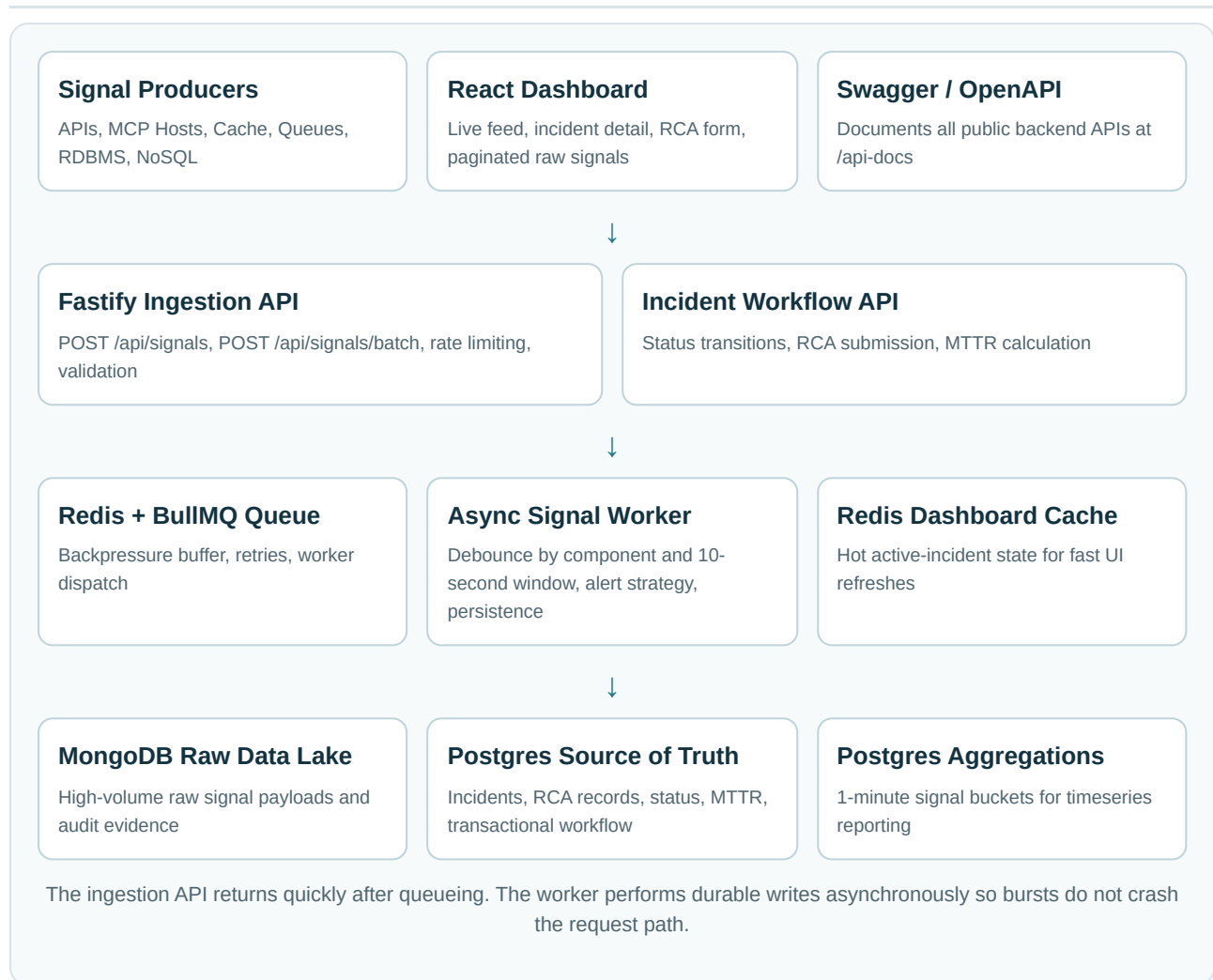
After cloning the GitHub repository, run:

```
docker compose up --build -d
```

Then open:

- Frontend Dashboard: <http://localhost:5173>
- Backend Health: <http://localhost:4000/health>
- Swagger API Docs: <http://localhost:4000/api-docs>
- OpenAPI JSON: <http://localhost:4000/api-docs/json>

3. Architecture Diagram



4. Technology Stack

Layer	Technology	Reason
Backend	Node.js, TypeScript, Fastify	Fast HTTP server, strong typing, simple API structure.
Async Processing	Redis, BullMQ	Queue-based backpressure, retries, worker concurrency.
Source of Truth	Postgres, Prisma	Transactional incident, RCA, workflow, and MTTR data.
Raw Data Lake	MongoDB	Flexible storage for high-volume raw signal payloads.
Hot Path Cache	Redis	Active incident dashboard cache and queue storage.
Frontend	React, Vite	Responsive operational dashboard with fast local development.
Packaging	Docker Compose	One-command local environment for reviewer execution.

5. Main Features Implemented

- High-throughput signal ingestion API.
- Async queue and worker-based persistence.
- Debouncing: repeated component signals in a 10-second window create one incident.
- Raw signal audit log stored in MongoDB.
- Structured incidents, RCA, and MTTR stored in Postgres.
- Redis dashboard cache to avoid frequent source-of-truth scans.
- Alert Strategy Pattern for severity and responder assignment.
- State Pattern for incident lifecycle transitions.
- Mandatory RCA before closure.
- Paginated raw signals in UI to avoid browser overload.
- Swagger/OpenAPI documentation for every public API.
- Health endpoint and throughput metrics printed every 5 seconds.

6. API Documentation

Swagger UI exposes every route with method, purpose, request body, query params, and response shape.

Endpoint	Purpose
GET /health	Checks backend health.
POST /api/signals	Ingests one signal.
POST /api/signals/batch	Ingests up to 5000 signals.
GET /api/incidents	Lists active incidents from Redis hot cache.
GET /api/incidents/:id	Returns one incident with RCA.
GET /api/incidents/:id/signals	Returns paginated raw signals from MongoDB.

<code>PATCH /api/incidents/:id/status</code>	Transitions incident status.
<code>POST /api/incidents/:id/rca</code>	Submits RCA and calculates MTTR.
<code>GET /api/metrics/aggregations</code>	Returns recent aggregation buckets.

7. Backpressure and Resilience

The ingestion API validates and enqueues signals, then returns `202 Accepted`. Database writes happen in the worker. This protects the request path when Postgres or MongoDB is slow.

- Redis/BullMQ queue absorbs bursts.
- Worker concurrency is configurable with `WORKER_CONCURRENCY`.
- Failed persistence is retried with bounded exponential backoff.
- Rate limiting protects ingestion endpoints.
- Dashboard reads active incidents from Redis cache.
- Raw signals are paginated to avoid UI crashes on large incidents.

8. Live Feed Design

The live feed currently uses polling instead of WebSocket. The frontend calls `GET /api/incidents` every 5 seconds. The backend serves that response from Redis hot dashboard cache.

This is reliable for the assignment and avoids long-lived socket complexity. A future production upgrade could replace polling with Server-Sent Events or WebSocket push from the backend.

9. Workflow and RCA

Allowed lifecycle:

```
OPEN -> INVESTIGATING -> RESOLVED -> CLOSED
```

- Invalid transitions return conflict errors.
- Closing without RCA is rejected.
- RCA requires start time, end time, category, fix applied, and prevention steps.
- MTTR is calculated from first signal time to RCA end time.

10. Testing and Verification

Commands used for verification:

```
npm test
npm run build
docker compose down
docker compose up --build -d
docker compose ps
docker compose exec -T backend npm run simulate:failure
```

Automated tests cover:

- RCA validation.
- Cannot close without RCA.

- Valid RCA allows closure.
- Invalid status transitions.
- Alert severity mapping for RDBMS and cache.

11. Bonus / Non-Functional Items

- Swagger/OpenAPI documentation.
- Dockerized one-command setup.
- Backpressure through Redis queue.
- Rate limiting on ingestion.
- Retry logic for persistence path.
- Health endpoint.
- Throughput metrics every 5 seconds.
- Paginated raw signal evidence.
- Docker build hygiene through `.dockerignore`.
- Readable README and implementation guide.

12. Submission Checklist

- Codebase is present in GitHub.
- Docker Compose setup is present.
- README includes architecture, setup, and backpressure explanation.
- Swagger documents all APIs.
- Sample failure simulation is included.
- Markdown docs and implementation guide are checked in.
- PDF includes the GitHub repository link before final submission.